



**UNIVERSIDAD
DE GRANADA**

**E.T.S. DE INGENIERÍAS INFORMÁTICA Y DE
TELECOMUNICACIÓN**

**Departamento de Ciencias de la
Computación e Inteligencia Artificial**

Algorítmica, Grupo C

Guión de Prácticas

Práctica 3: Algoritmos Voraces (Greedy)

Curso 2025-2026

Grado en Informática

Objetivo

El objetivo de esta práctica es que el estudiante aprecie la utilidad de los métodos voraces (greedy) para resolver problemas de forma muy eficiente, en algunos casos obteniendo soluciones óptimas y en otros aproximaciones. Para ello cada equipo de estudiantes deberá resolver uno de los problemas (asignado al azar) que se describen en la sección 1. En todos los casos se debe especificar la formulación del problema dentro del esquema general de los algoritmos voraces (explicando además por qué la estrategia elegida parece intuitivamente razonable), también demostrar la optimalidad del algoritmo (o al menos dar argumentos o indicios sólidos que sugieran esa optimalidad) e implementarlo. Adicionalmente, todos los equipos deberán diseñar e implementar algoritmos voraces (como heurística) para resolver el problema descrito en la sección 2.

1. Problemas

1.1. Ratones

Supongamos que hay n ratones y n agujeros a lo largo de una línea. En cada agujero solo cabe un ratón. Un ratón puede permanecer en su posición, moverse un paso a la derecha, de x a $x + 1$, o moverse un paso a la izquierda, de x a $x - 1$. Cualquiera de estos movimientos consume 1 segundo. Los ratones pueden moverse simultáneamente. El objetivo es asignar los ratones a los agujeros de forma que el tiempo que tarda el último ratón en llegar a un agujero sea mínimo.

Por ejemplo, para $n = 3$ los ratones pueden estar en las posiciones 4, -4, 2 y los agujeros en las posiciones 4, 0, 5.

Diseñad e implementad (y analizad la eficiencia) un algoritmo voraz para resolver este problema (que devuelva la asignación de cada ratón a cada agujero y el tiempo que tardan en resguardarse todos) y demostrad su optimalidad. Proponed e implementad también algún otro algoritmo (greedy o no) y comparadlos.

1.2. Análisis químico

Un técnico de laboratorio quiere averiguar la composición química de un asteroide. Para ello, dispara láseres a distintas frecuencias al asteroide. Cada sustancia s_i de una lista de n posibles sustancias constituyentes tiene un rango conocido $[l_i, h_i]$ en el que la sustancia reaccionará al láser (los rangos pueden estar solapados). El objetivo es elegir el conjunto más pequeño de frecuencias $f_1, \dots, f_k$ para que cada sustancia s_i reaccione a una de estas, es decir, para cada $i = 1, \dots, n$ hay al menos un j con $l_i \leq f_j \leq h_i$.

Diseñad e implementad (y analizad la eficiencia) un algoritmo voraz para resolver este problema (que devuelva las frecuencias elegidas y su número) y demostrad su optimalidad. Proponed e implementad también algún otro algoritmo (greedy o no) y comparadlos.

1.3. Gestión editorial

Una editorial científica debe gestionar la publicación de n artículos en una plataforma digital. Cada artículo debe pasar por dos fases consecutivas:

- Revisión editorial centralizada: cada artículo i debe ser evaluado por un comité editorial único, que solo puede revisar un artículo a la vez. Esta fase requiere $p(i)$ unidades de tiempo para cada artículo, $i = 1, \dots, n$.
- Maquetación y publicación: una vez aprobada la revisión, el artículo pasa a un equipo de maquetación. La editorial dispone de un número suficientemente grande de maquetadores, de modo que esta fase puede realizarse en paralelo para todos los artículos. Esta fase requiere $f(i)$ unidades de tiempo, para cada artículo, $i = 1, \dots, n$.

El proceso completo finaliza cuando *todos* los artículos han sido publicados. La editorial debe decidir cómo proceder, con el objetivo de minimizar el tiempo total hasta que el último artículo quede publicado.

Diseñad e implementad (y analizad la eficiencia) un algoritmo voraz para resolver este problema (que devuelva la solución y el valor de tiempo obtenido) y demostrad su optimalidad. Proponed e implementad también algún otro algoritmo (greedy o no) y comparadlos.

2. Cámaras de vigilancia

Se desea garantizar la seguridad de un complejo mediante la instalación de cámaras de vigilancia. Cada intersección entre pasillos está numerada del 1 al n , y se conoce también el conjunto de pasillos que unen intersecciones mediante pares (i, j) , del pasillo que une la intersección i con la intersección j . Una cámara instalada en una intersección puede vigilar todos los pasillos que llegan a ella. Por ejemplo, si existen 7 intersecciones y 7 pasillos $(1, 2), (1, 3), (3, 4), (3, 5), (4, 6), (5, 6), (6, 7)$, entonces colocar cámaras en las intersecciones $\{1, 3, 6\}$ nos permite tener todos los pasillos vigilados (también si instalamos las cámaras en las intersecciones $\{1, 4, 5, 7\}$, por ejemplo). El objetivo es conseguir instalar el menor número posible de cámaras de modo que todos los pasillos estén vigilados por, al menos, una cámara.

2.1. Tareas a realizar

- Diseñad (dando una explicación detallada) e implementad *al menos* un algoritmo voraz para resolver el problema anterior, que devuelva las localizaciones escogidas para las cámaras, y el número de estas. Haced un estudio de su eficiencia (teórico, empírico e híbrido). Proporcionad algún contraejemplo de que el algoritmo no encuentra siempre la solución óptima.
- Para evaluar si los algoritmos voraces que se propongan se acercan más o menos a la solución óptima, se pueden comparar sus resultados con los de la verdadera solución óptima, obtenida mediante una evaluación exhaustiva de todas las posibilidades (y quedándonos con la de mejor valor). Para ello se proporciona un programa de fuerza bruta (`camaras-fb.cpp`) que realiza dicha tarea (pero que solo puede emplearse tardando un

tiempo razonable en la experimentación con problemas de tamaño reducido, hasta 28, tardando unos 30 segundos). Para esos tamaños pequeños haced un estudio comparativo de los resultados obtenidos con los de la verdadera solución óptima (no en términos de eficiencia, sino de efectividad). Como los resultados pueden variar mucho dependiendo de la instancia del problema, probad con diferentes instancias de cada tamaño.

- OPCIONALMENTE, diseñad un algoritmo voraz (que sea óptimo) para el caso en que la forma del complejo tenga estructura de árbol.

Material a entregar

Se debe entregar una memoria *detallada* con todas las tareas realizadas (de las dos partes de la práctica), así como el código de *todos* los programas desarrollados. En particular, debe explicarse con claridad y detalle el diseño de los algoritmos. El informe debe entregarse en formato pdf. En el informe deben aparecer los nombres de todos los miembros del equipo que hayan participado en la realización de la práctica, así como el porcentaje de participación de cada uno de ellos.